

Roadmap do projeto de detecção de trojans

Márcio Araújo · Grupo 10 IAAC · 7 de outubro de 2026

A componente técnica do dataset Trojan está funcional como demonstração local, com pipeline, modelos, API, notebook e agentes testados. O modelo não atinge o critério de produção. A entrega acadêmica completa depende ainda dos restantes datasets, dos templates e do trabalho do grupo.

Objetivo e fontes

Este plano adapta o roadmap das aulas de 18 e 21 de setembro ao canvas fornecido, ao ficheiro Trojan_Detection.csv e aos critérios de avaliação. A execução usa CRISP ML para o ciclo técnico e uma proposta SCRUM para organização. Não regista reuniões retroativas.

O que já existia

Componente	Estado encontrado
Negócio e dados	Canvas preenchido e dataset disponível com 177 482 fluxos e 86 colunas.
Código e experiências	Seis scripts, CLI, gráficos e resultados anteriores; notebook vazio.
GitHub	Branch Márcio com exemplo FastAPI de outro problema, preservado na raiz.
Validação	Resultados antigos por reproduzir; ausência de testes, API Trojan e Git local.

Correções principais

A inferência exigia Class e recalculava medianas no lote recebido. A preparação aprendia medianas antes da separação e os limiares eram escolhidos no teste. A nova versão aprende preparação apenas no treino, fixa modelo e limiar na validação e avalia uma reserva separada por IP de origem.

Também foram corrigidos o tratamento de extremos no PSI e a medição de latência, que agora usa chamadas individuais com transformação e inferência. As experiências antigas estão preservadas numa cópia de segurança e não validam a versão atual.

Roadmap adaptado e evidências

Cada etapa termina com um artefacto verificável. A conclusão técnica é distinta da aprovação de desempenho e da entrega do grupo.

Etapa	Trabalho e evidência
1 Negócio	SOC como utilizador, classificação no fim do fluxo, custos FP/FN e critério original. docs/canvas_revisto.md.
2 Compreender dados	Qualidade, classes por dia, estatística, Pearson, Spearman, PCA e assinaturas. reports/qualidade_dados.json e figs/.
3 Preparar dados	Esquema fixo, retirar IDs, log de taxas, categorias, mediana só no treino e variância. src/preparacao.py.
4 Treinar modelos	Baseline, logística, árvore, RF, HGB, regularização e ablação. models/ e comparacao_validacao.csv.
5 Avaliar	Grupos de treino, validação e teste disjuntos por Source IP; limiar na validação; teste final e bootstrap. avaliacao_final.json.
6 Disponibilizar	Pacote completo, CLI sem rótulos, API, dashboard e notebook. predict.py, api.py e notebooks/01_pipeline.ipynb.
7 Monitorizar	PSI, métricas por dia, latência e plano para dados recentes. reports/ e docs/operacao.md.
8 Integrar agentes	Classificador factual e gerador Qwen local, com orquestração Python e LangGraph. src/agentes.py.
9 Entregar	Testes, CI, documentação do código, model card, diagramas e commits. docs/, tests/ e Git.

Organização SCRUM proposta

Sprint 1: negócio e EDA. Sprint 2: preparação e comparação de modelos. Sprint 3: avaliação e interpretação. Sprint 4: API, notebook e monitorização. Sprint 5: agentes, documentação e integração do grupo. O backlog inclui critérios de aceitação e o responsável conhecido; a equipa deve preencher as reuniões reais nos seus templates.

Resultados e decisão sobre o modelo

Conjunto	Fluxos	Benign	Trojan	IPs origem
Treino	60 693	12 858	47 835	1 008
Validação	111 185	71 013	40 172	337
Teste	5 604	2 928	2 676	449

Zero IPs de origem partilhados entre conjuntos. As frações são de grupos, não de linhas; poucos hosts dominam o dataset. IPs de destino e padrões podem repetir. O teste temporal recente contém só positivos e não permite estimar FPR ou AUC ROC.

Métrica no teste	Resultado
Modelo escolhido na validação	HistGradientBoosting
AUC ROC	0,6425
Recall Trojan	4,30%
Taxa de falsos positivos	4,23%
Precisão dos alertas	48,12%
Limiar fixado na validação	0,927568
IC95% AUC por bootstrap de IPs	0,532 a 0,768

Critério original não atingido

O canvas exige recall de pelo menos 90% com FPR de no máximo 5%. O teste ficou muito abaixo da meta de recall. O sistema serve para demonstração e revisão humana; não executa bloqueio automático. Reduzir a meta não tornaria esta avaliação uma aprovação do critério original.

A latência medida foi 88,6 ms de média e 97,7 ms no percentil 95, em cem chamadas individuais. Inclui transformação de estatísticas já extraídas, imputação e inferência. Não inclui PCAP nem garante o pior caso.

A auditoria atual encontrou 650 grupos de features idênticas com rótulos distintos, envolvendo 3 331 linhas. Não reproduziu a afirmação histórica de 61,3%. Esta observação não prova erro de rótulo sem investigar o contexto temporal.

Entregas e passos finais

Feito e verificado

20 testes aprovados, sem falhas ou skips. Notebook com seis células de código executadas. CLI com cinco fluxos sem rótulo. API e interface testadas com um fluxo real. Dois sistemas de agentes executados com Qwen local e factos equivalentes. Dependências sem conflitos e versões fixadas.

O que ainda depende do grupo

Pendência	Condição de conclusão
Datasets e equipa	Fornecer os outros datasets/modelos e identificar os restantes elementos.
Professor e templates	Confirmar validação do dataset, origem/licença e modelos do Classroom.
SCRUM e relatório académico	Registar reuniões reais e redigir 25 a 30 páginas no template, com autoria dos alunos e histórico partilhado.
Defesa individual	Preparar 5 a 7 minutos e explicar os componentes de código escolhidos.
Uso operacional	Novos dados recentes com ambas as classes, nova reserva, metas atingidas e capacidade do SOC confirmada.

Executar no VS Code

A pasta de trabalho é D:\Transferencias\projeto-trojan-detection\projeto. O interpretador é .venv\Scripts\python.exe. O roadmap, README e tarefas do VS Code descrevem a sequência completa. A API local está disponível em <http://127.0.0.1:8000> enquanto o processo estiver ativo.

Comandos no terminal do projeto

```
..\venv\Scripts\python.exe -m pytest -q
..\venv\Scripts\python.exe predict.py examples/fluxos_sem_rotulo.csv
..\venv\Scripts\python.exe -m uvicorn api:app --host 127.0.0.1 --port 8000
```

GitHub: Marinho050/Grupo10-iaac, branch Márcio. Os classificadores e manifestos são versionados; o dataset, o ambiente e os pesos Qwen ficam fora do Git. O download do gerador usa a revisão guardada no manifesto. A CI foi aprovada no GitHub para a versão de código e3d4823, incluindo dependências, testes centrais e CLI. O Dockerfile é fornecido, mas Docker não está instalado nesta máquina.